A One Round Protocol for Tripartite Diffie–Hellman

Review

在本科时候，我们学习过椭圆曲线的点加与标量乘操作。设椭圆曲线：

$$E: y^2 = x^3 + ax + b$$

则对于曲线上的两点 $P = (x_1, y_1), Q = (x_2, y_2)$，我们定义：

$$\lambda = \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1}, & P \neq Q \\ \dfrac{3x_1^2 + a}{2y_1}, & \text{otherwise} \end{cases}$$

于是我们得到 $P + Q$ 的坐标 (x_3, y_3) 满足：

$$\begin{cases} x_3 = \lambda^2 - x_1 - x_2 \\ y_3 = \lambda(x_1 - x_3) - y_1 \end{cases}$$

无穷远点 $\mathcal{O}$ 为单位元：

$$P + (-P) = \mathcal{O}, P + \mathcal{O} = P$$

What plain ECC cannot do

我们现在知道，普通 ECC 可以做点加，以及标量乘操作。这意味着我们可以用椭圆曲线做两方的 Diffie-Hellman 协议，过程如下：

- Alice 和 Bob 确定椭圆曲线 E 和公共参数 P，Alice 持有私钥 a，Bob 持有 私钥 b；
- Alice 计算并公开 aP，Bob 计算并公开 bP；
- Alice 计算 $a(bP) = abP$，Bob 计算 $b(aP) = abP$，最终双方都得到相同密钥 abP.

这样，我们就实现了在一轮内完成密钥交换。

但假如现在有 Alice，Bob，Carol 三人呢?

假如我们按照以前的方法，每个人生成对应的 aP, bP, cP，那么 Alice 会拥有 a, bP, cP，仅仅靠点加和标量乘操作，并不能构造出 f 满足 $f(bP, cP) = bcP$——实际上这就是 Computational Diffie-Hellman (CDH) 问题本身。

如果我们换一种办法，比如说 Bob 和 Carol 已经通过密钥交换得到了 bcP，那么他们中的任意一个人又可以与 Alice 再进行一次 ECDH 协议得到最终的 $abcP$。这样也是可行的，但问题是，轮数就从 1 变成了 2，不再满足 one-round 的目标。

Bilinear pairing

那我们需要什么新工具，才能在一轮之内，绕过先前的 CDH 假设，完成三人的 key exchange 呢?

换句话说，Alice 可以拿到来自 Bob 的 bP 和 Carol 的 cP，她现在需要一种*变换*，可以把 bP, cP 转化成一种既与 P 有关，也与 bc 有关的一种形式。

再具体一点，这样的工具，需要满足类似

$$f(bP, cP) \stackrel{?}{=} F(P, bc)$$

的一种结构。而满足这种需求的一个自然候选，就是双线性映射（bilinear pairing）。

正如 ECC 是 19 世纪椭圆曲线理论在密码学中的应用，Joux 也把一个经典的双线性数学构造应用到了三方密钥交换中。

而这个构造，或者前文所说的*变换*，就是——weil pairing。而对应的工作，就是 Antoine Joux 的经典论文 *A One Round Protocol for Tripartite Diffie–Hellman*.

Weil Pairing

为了建立一个大体的直觉，我们先不深入 weil pairing 的具体细节，假设的确存在一个函数 $e_m(P,Q)$，这里 P,Q 是椭圆曲线 E 的两个点，函数满足以下性质：

> 注意 $e_m(P,Q)$ 的取值不再是椭圆曲线上的点，而是某个乘法群中的 m 次单位根。

- 单位根：$e_m(P,Q)^m = 1$
- 双线性：$e_m(P_1+P_2,Q) = e_m(P_1,Q)e_m(P_2,Q)$，$e_m(P,Q_1+Q_2) = e_m(P,Q_1)e_m(P,Q_2)$
- 非退化性：$e_m(P,Q) = 1\ \forall P \in E[m] \Rightarrow Q = \mathcal{O}$
- 交错性：$e_m(P,Q) = e_m(Q,P)^{-1}, e_m(P,P) = 1.$

中间的双线性稍微变形一下，就可得到：

$$e_m(aP,bQ) = e_m(P,Q)^{ab}$$

这刚好符合之前我们渴望的函数 f，问题解决了一半。

The true difficulty

接下来的另一半问题是，括号里面的 P, Q 具体应该写成什么。如果 $P = Q$ 的话，根据上一页的交错性性质，最终的结果总为1，这肯定是行不通的。

然后我们也可以发现当 $P = kQ$ 时也会得到常数结果1，这意味着 P, Q 是线性无关的，这意味着 Joux 的正式协议必须要让每方同时广播两个毫不相干的点。

现在总结一下，我们的直觉，离最终的正式协议，还差了一下几点：

- 不能直接用 $e(P, P)$，因为交错性会使结果平凡；
- 因此需要两个独立方向，而不是单一基点 P；
- 但又需要一种对称的公开信息组织方式，使三方都能计算到同一个共享值。

而真正困难的 insight 恰恰在这里——即使意识到以上三点，Joux 的最终组织方式仍然并不显然。也就是说，pairing 提供了“把指数耦合起来”的能力，但怎样把这种能力嵌入一个真正的一轮三方协议，仍然需要额外的设计。

大家可以想 30 秒，然后再继续。

Joux's insight

这里 Joux 的 insight 就是：不再让每个人只广播单个点的倍数，而是让每一方都广播一对经过特殊组织的公开元素。

我们仍然假设 P, Q 是椭圆曲线上线性无关的两个点（已经公开），我们令：

- Alice 广播 (aP, aQ)；
- Bob 广播 (bP, bQ)；
- Carol 广播 (cP, cQ).

那么在下一轮之后，Alice 将会拿到 (bP, bQ, cP, cQ)，加上 Alice 本身也知道 (a, P, Q)，她的任务就是要想办法从 (bP, bQ, cP, cQ) 中组合出一种关于 bc 的量——

大家想到了吗？之前的 weil pairing 就可以排上用场了，因为 $e(bP, cQ) = e(P, Q)^{bc}$，再加上 alice 的私钥 a，我们最终就可以得到共有的结果 $e(P, Q)^{abc}$.

这一步对于 Bob 和 Carol 同样如此。

综上，我们便在一轮之内实现了三方的密钥交换。

Protocol

- Setup:

公开椭圆曲线 $E(\mathbb{F}_p)$，两个位于 E 上且线性无关的点 P, Q.

- Keygen:

Alice，Bob 和 Carol 分别准备自己的私钥 $a, b, c \in \mathbb{F}_p$.

- Reveal:

Alice 生成公钥 (aP, aQ)，Bob 生成 $(bP，bQ)$，Carol 生成 (cP, cQ)，公开这些公钥。

- Compute shared key:

  - Alice 计算 $e(bP, cQ)^a$;
  - Bob 计算 $e(aP, cQ)^b$;
  - Carol 计算 $e(aP, bQ)^c$;

最终三人都得到共享密钥 $K = e(P, Q)^{abc}$，协议结束。

Security Intuition

考虑三个攻击方向：内部半诚实、外部窃听，以及 MITM：

1. 对于第一条，我们假设攻击者就是 Alice（假如她是半诚实的）：

Alice 在第一轮之后，本身拥有 $(a, P, Q, bP, bQ, cP, cQ)$

- 但由于 ECDLP，Alice 不能拿到私钥 b, c；
- 由于 DLP，Alice 即使知道 $e(P, Q)$ 和 $e(P, Q)^{abc}$，也拿不到 bc，自然也拿不到 b, c。

更进一步来说，假如 Alice 与 Bob 串通，ECDLP 与 DLP 也仍然不能让 Alice 和 Bob 拿到私钥 c。

2. 对于第二条，外部攻击者 Eve 只可以拿到 (aP, aQ, bP, bQ, cP, cQ)，攻击者想知道 $e(P, Q)^{abc}$——这是一个 tripartite / Bilinear Diffie–Hellman（BDH）难题，我们有以下困难程度层级：

$$BDH \leq CDH \leq DLP$$

也就是如果你有解决 DLP 的方法，你就可以解决 CDH 问题，进而解决 BDH 问题。

3. 对于第三条，答案是与经典 Diffie-Hellman 相同，不防 MITM——具体攻击方法留作课后习题。

Zeros and poles

在讲 weil pairing 的实现之前，首先我们介绍一下有理函数和除子 div 的概念：

对于任意单变量有理函数：

$$f(X)=\frac{a_0+a_1X+a_2X^2+\cdots+a_nX^n}{b_0+b_1X+b_2X^2+\cdots+b_mX^m}$$

根据代数基本定理，我们总可以在复平面上将其因式分解为：

$$f(X)=\frac{a(X-\alpha_1)^{e_1}(X-\alpha_2)^{e_2}\cdots(X-\alpha_r)^{e_r}}{b(X-\beta_1)^{d_1}(X-\beta_2)^{d_2}\cdots(X-\beta_s)^{d_s}}$$

这里 α_i,β_j 共 $r+s$ 个数互不相同。

为了简便，我们可以使用除子将式子简化：

$$\mathrm{div}(f)=e_1[\alpha_1]+e_2[\alpha_2]+\cdots+e_r[\alpha_r]-d_1[\beta_1]-d_2[\beta_2]-\cdots-d_s[\beta_s]$$

这里的α_i被称为 zeros（零点），β_i 被称为 poles（极点），而前面的系数e_i,d_i则被称为 multiplicity（重数）。

当然，这里的 X 大写就意味着 f 的对象不一定只能是一个数，X 的本身也可以由多变量的函数构成，因此 f 也变成了多变量函数。进一步地，如果$f = f(x, y)$ 中的 (x, y) 由椭圆曲线 E 所约束，那么 E 中所对应的点有的可以让分子为零，有的可以让分母为零，因此这些点也可以被分别称为零点和极点。

> e.g. 我们令$E: y^2 = (x-\alpha_1)(x-\alpha_2)(x-\alpha_3), P \in E, f(P) = f(x, y) = y.$
> 那么 $P_1, P_2, P_3 = (\alpha_1, 0), (\alpha_2, 0), (\alpha_3, 0)$ 肯定在 E 上。因此：
>
> $$\mathrm{div}(f) = [P_1] + [P_2] + [P_3] - 3[\mathcal{O}]$$
>
> 这里$[P_1] + [P_2] + [P_3]$ 很好理解，因为此时 $f(x, y) = y = 0$。虽然$f(x, y)$ 并没有分母，但极点$\mathcal{O}$的重数是可以通过射影变换与无穷小分析证明是3的，这里篇幅原因不做展开。

经过上述例子我们可以猜想 E 上 $\mathrm{div}(f)$ 的一些性质：

1. $\mathrm{div}(f) = \mathrm{div}(g)$，当且仅当存在常数$c$，$f = cg$.
2. 展开式系数（度数）之和为0.
3. 如果对这几个点做点加运算，结果为 $\mathcal{O}$.
4. 特别地，如果不存在零点和极点，那么 f 为常数。

前者被称为**除子的唯一性定理**，后三者被称为**主除子定理**。结论与有理函数f的选取无关。

Line fuction

基础知识铺垫完成，现在引入 miller 算法中位于核心地位的 line function $g_{P,Q}$：

$$g_{P,Q} = \begin{cases} \frac{y-y_P-\lambda(x-x_P)}{x+x_P+x_Q-\lambda^2} & (\lambda \neq \infty) \\ x - x_P & \text{else} \end{cases}$$

你可以感性地理解成用过点 $P, Q, -(P+Q)$ 的直线去除过 $P+Q, -(P+Q)$ 的直线。因此可以写出对应的除子：

$$\mathrm{div}(g_{P,Q}) = [P] + [Q] - [P+Q] - [\mathcal{O}]$$

在代码实现方面，我们可以把 line function 抽象为一个关于椭圆曲线任意三点 P, Q, S 的函数，具体实现如下（这里的 `self, other, S` 都是实例化的椭圆曲线类中的点，分别对应点 P, Q, S）：

```
    def lineFunc(self, other, S) -> int: # P, Q, S. S is the f_p(S)
        xp, xq, yp, yq = self.x, other.x, self.y, other.y

        if self.calcLam(other) == math.inf:
            return S.x - xp
        lam = self.calcLam(other)
        return self.frac(S.y - yp - lam*(S.x - xp), S.x + xp + xq - lam*lam)
```

Miller's Algorithm

设正整数 m 的二进制表示为 $m = \overline{b_{n-1}b_{n-2}...b_0}, b_i \in \{0,1\}, b_{n-1} = 1$，以下算法可以生成一个有理函数 f_P，使得 $\mathrm{div}(f_P) = m[P] - [mP] - (m-1)[\mathcal{O}]$：（证明见后页）

```
T = P, f = 1
for i from (n-2) to 0:
    f = f^2 * g_{T,T}
    T = 2T
    if b[i] == 1:
        f = f * g_{T,P}
        T = T + P
return f
```

> 注：这里的 $g_{T,T}$ 就是上一页提到的 Line function.

特别的，如果点 $P \in E[m]$（即 P 满足 $mP = \mathcal{O}$），那么 $\mathrm{div}(f_P) = m[P] - m[\mathcal{O}]$，符合之后要提到的 weil pairing 的条件。

我们这里对比一下伪代码以及对应的具体实现：

```
T = P, f = 1
for i from (n-2) to 0:
    f = f^2 * g_{T,T}
    T = 2T
    if b[i] == 1:
        f = f * g_{T,P}
        T = T + P
return f
```

```
    def miller(self, S) -> int: # P, S, calc f_P(S)
        if self.O:
            return 1
        T = self; f = 1
        n = bin(self.calcOrder())[3:] # 0b1 01001..101

        for bit in n:
            f = f * f * T.lineFunc(T, S) % self.mod
            T = T.add(T)
            if bit == '1':
                f = f * T.lineFunc(self, S) % self.mod
                T = T.add(self)
        return f
```

Correctness proof of Miller's algorithm

使用归纳法证明如下：

首先 $m_1 = b_{n-1}$ 时，算法返回 1 且 $T = P$，1是常数，既没有零点也没有极点。将$m = 1$ 代入 $\mathrm{div}(f_P)$，所有的项都消掉了，因此 $m_1 = b_{n-1}$ 成立。
设 $m_i = \overline{b_{n-1}b_{n-2}...b_{n-i}}$。有 $T = m_iP$，$\mathrm{div}(f_P) = m_i[P] - [m_iP] - (m_i - 1)[\mathcal{O}]$ 成立。我们需要证明 $m_{i+1} = \overline{b_{n-1}b_{n-2}...b_{n-i-1}}$ 时的情况成立：
假设 $b_{n-i-1} = 0$，则不走 if 分支，我们实际的运算为 $f' = f^2 \cdot g_{m_iP,m_iP}, m_iP = 2m_iP$。则此时新的 $\mathrm{div}(f')$：

$$= 2(m_i[P] - [m_iP] - (m_i - 1)[\mathcal{O}]) + ([m_iP] + [m_iP] - [2m_iP] - [\mathcal{O}])$$

$$= m_{i+1}[P] - 2[m_iP] - 2(m_i - 1)[\mathcal{O}] + 2[m_iP] - [m_{i+1}]P - [\mathcal{O}]$$

$$= m_{i+1}[P] - [m_{i+1}]P - (m_{i+1} - 1)[\mathcal{O}]$$

结果正确，并且此时 $T' = 2T = 2m_iP = m_{i+1}P$，也符合归纳要求，因此归纳在 $b_{n-i-1} = 0$ 分支下成立。同理也可证得 $b_{n-i-1} = 1$ 分支下成立。
综上所述，该算法可以生成有理函数 f_P, 使得 $\mathrm{div}(f_P) = m[P] - [mP] - (m - 1)[\mathcal{O}]$。Q.E.D

The true definition of Weil Pairing

我们设有理函数 f_P, f_Q 满足 $\mathrm{div}(f_P) = m[P] - m[\mathcal{O}], \mathrm{div}(f_Q) = m[Q] - m[\mathcal{O}]$。假设$P, Q \in E[m]$，且 S 不为 P, Q 的线性组合，则定义 weil pairing：

$$e_m(P, Q) = \frac{f_P(Q+S)}{f_P(S)} / \frac{f_Q(P-S)}{f_Q(-S)}$$

前文说过，weil pairing 的值仅仅与 P, Q 的坐标有关，与有理函数 f_P, f_Q 以及 S 的选取均无关。

由于我们已经使用 miller 算法计算出了 $f_P(S)$，因此我们可以直接通过 miller 类实现 weil 类：

```
    def weil(self, Q, S) -> int: # P, Q, S
        negS = EpPoint(S.x, -S.y, self.curve)    # -S

        res1 = self.frac(self.miller(Q.add(S)), self.miller(S))
        res2 = self.frac(Q.miller(self.add(negS)), Q.miller(negS))

        return self.frac(res1, res2)
```

举一个具体的例子，我们假设存在一个定义在 $\mathbb{F}_{631}$ 上的椭圆曲线 E：

$$y^2 = x^3 + 30x + 34$$

并取点 P, Q, S 分别为 $(36, 60), (121, 387), (0, 36)$。大家可以分别验证一下他们都在 E 上，并且阶分别为 $5, 5, 130$.

这里的 P 与 Q “线性无关”，而 S 不在 P 与 Q 的“线性组合”内（即找不到 m, n 使得 $S = mP + nQ$），因此可以用来进行 weil pairing 的计算。

```
print('P, Q\'s weil pairing:', P.weil(Q, S))
# P, Q's weil pairing: 242
print(242**5 % 631)
# 1
print('Q, P\'s weil pairing:', Q.weil(P, S))
# Q, P's weil pairing: 279
print(242*279 % 631)
# 1
```

可见，性质一（单位根，这里顺便也可以说明 $e_m(P, Q)$ 中的 $m = 5$）与性质四（交错性）成立。

然后我们再举另一个例子：

```
P3 = P.mul(3)
Q4 = Q.mul(4)
print('P3, Q4\'s weil pairing:', P3.weil(Q4, S))
# P3, Q4's weil pairing: 512
print(242**12 % 631)
# 512
```

便说明了性质二（双线性性）。

举例说明性质三也很简单：

```
O = P.mul(5)
print('P, O\'s weil pairing:', P.weil(O, S))
# P, O's weil pairing: 1
```

也就是非退化性成立。

这里再强调一点，weil pairing 计算的值，仅仅与 $e_m(P,Q)$ 中的点 P,Q 与椭圆曲线 E 的具体取值有关，而与 line function 的选择，和代码为了辅助计算的点 S 均无关。

Tate Pairing

当然，weil pairing 在数学上的确十分优雅，但在定义上较为繁琐，不利于密码学的高效实现。于是实际上用得更多的是 Tate pairing 以及对应的工程实现（reduced Tate / ate / optimal ate pairing）。这里介绍第一种：

Tate pairing 与 weil pairing 的区别是其椭圆曲线定义在有限域 $\mathbb{F}_q$ 上，而后者可以是任意域。我们定义：

$$\hat{\tau}(P,Q) = (\frac{f_P(Q+S)}{f_P(S)})^{(q-1)/l} \in \mathbb{F}_q$$

其中 l 是质数， $P \in E(\mathbb{F}_q)[l]$， $Q \in E(\mathbb{F}_q), q \equiv 1 \pmod{l}$.

可以发现，tate pairing 的 Miller function 计算量比 weil pairing 少一半，并且仍满足**单位根**和**双线性**两个性质，所以在密码学中更受青睐。例如 tate pairing 的变种 ate pairing 在以太坊中被广泛使用，而 CHES 上现在仍有针对 optimal ate pairing 的优化工作。https://eprint.iacr.org/2025/1283

```
    def tate(self, Q, S) -> int:
        q = self.mod; l = self.calcOrder()
        if q % l != 1:
            raise ValueError('q and l don\'t support q == 1 mod l')

        tau = self.frac(self.miller(Q.add(S)), self.miller(S))
        return pow(tau, self.frac(q-1, l), q)
```

这里验证一下 reduced Tate 的两个性质（单位根、双线性性）：

```
print('P, Q\'s tate pairing:', P.tate(Q, S))
# P, Q's tate pairing: 279
print('Q, P\'s tate pairing:', Q.tate(P, S))
# Q, P's tate pairing: 228
```

显然 279 与 228 都是 $\mod 631$ 的五次单位根，满足单位根性质；但 $279 \times 228 \neq 1$，因此交错性不满足。

然后看一下双线性：

```
P3 = P.mul(3)
Q2 = Q.mul(2)

print('P, Q\'s tate pairing:', P.tate(Q, S))
# P, Q's tate pairing: 279
print('3P, 2Q\'s tate pairing:', P3.tate(Q2, S))
# 3P, 2Q's tate pairing: 279
```

因为 $279^6 = 279^{6\%5} = 279$，因此满足双线性性质。